

KV Store Distribuito con Vector Clock

Versioni causali, conflitti e convergenza

Architetture dei Sistemi Distribuiti

Lezione 23

Abbiamo studiato i vector clock come strumento per distinguere causalità e concorrenza.

Oggi li integriamo in un sistema concreto:

un KV store distribuito multi-master

La domanda non è più solo teorica:

quale contratto può offrire l'interfaccia quando esistono scritture concorrenti?

- ➊ problema: versioni scalari e scritture concorrenti;
- ➋ modello del laboratorio;
- ➌ file disponibili;
- ➍ ruoli nel sistema e in classe;
- ➎ version vector per chiave;
- ➏ dominio causale e concorrenza;
- ➐ siblings e risoluzione esplicita;
- ➑ comandi dell'interfaccia;
- ➒ demo guidata;
- ➓ safety, liveness e limiti.

Problema: Una Versione Scalare Non Basta

A> SET room aula-a

B> SET room aula-b

Le due scritture avvengono su repliche diverse, prima della sincronizzazione.

Domande:

- quale valore deve restituire GET room?
- una versione è più nuova dell'altra?
- scegliere l'ultimo messaggio ricevuto è corretto?
- cosa deve promettere l'interfaccia?

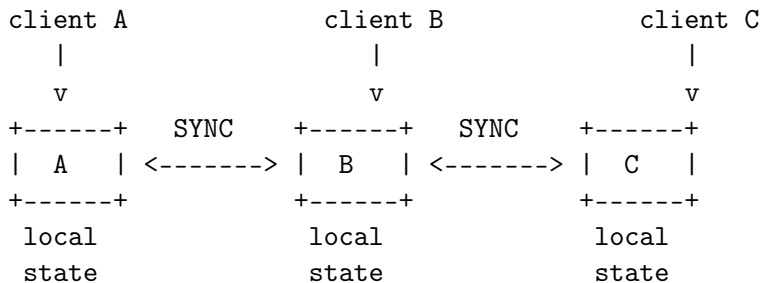
Usiamo tre repliche:

A, B, C

Ogni replica può:

- accettare scritture locali;
- leggere il proprio stato locale;
- sincronizzarsi con un'altra replica;
- rilevare versioni concorrenti;
- richiedere una risoluzione esplicita del conflitto.

Il laboratorio non nasconde il conflitto: lo rende parte del contratto.



SYNC è una sincronizzazione anti-entropy bidirezionale.

File	Ruolo
README.md	traccia rapida del laboratorio
vector_clock.py	primitive sui vector clock
node.py	replica KV multi-master
client.py	client interattivo
demo_vector_clock_kv.py	scenario automatico

Percorso:

labs/kv_store/vector_clock_replication/

Replica

- mantiene stato locale;
- produce aggiornamenti;
- incrementa la propria componente del clock;
- sincronizza snapshot con altre repliche.

Client

- parla con una replica specifica;
- non vede una memoria globale atomica.

Versione

- rappresenta un possibile valore di una chiave;
- contiene valore, clock, origine e flag di cancellazione.

Sibling

- è una versione concorrente della stessa chiave;
- non può essere scartata come vecchia.

Applicazione o operatore

- interpreta il conflitto;
- sceglie il valore con RESOLVE.

Struttura Di Una Versione

```
Version(  
    value="aula-a",  
    clock={"A": 1, "B": 0, "C": 0},  
    origin="A",  
    deleted=False,  
)
```

Ogni chiave può puntare a una lista di versioni:

```
data: key -> list[Version]
```

Una lista con più versioni vive rappresenta un conflitto.

Nel laboratorio il vector clock è associato alla versione della chiave.

Esempio:

```
value=aula-a clock=A:1,B:0,C:0
```

Significato:

- la versione include un aggiornamento prodotto da A;
- non include aggiornamenti prodotti da B;
- non include aggiornamenti prodotti da C.

Non è un timestamp fisico e non misura secondi.

Confronto Tra Clock

Dati due clock x e y :

$x \leq y$ se ogni componente di x
è $\leq$ della componente corrispondente di y

$x < y$ se $x \leq y$ e almeno una componente
è strettamente minore

Se $x < y$, allora y domina causalmente x .

old = A:1,B:0,C:0

new = A:1,B:1,C:0

new domina old:

- conosce tutto ciò che conosce old;
- contiene anche un aggiornamento prodotto da B.

La versione vecchia può essere compattata.

$v1 = A:1, B:0, C:0$

$v2 = A:0, B:1, C:0$

$v1$ non domina $v2$.

$v2$ non domina $v1$.

Conclusione:

le due versioni sono concorrenti

Quando due versioni sono concorrenti, il KV store mantiene entrambe.

room:

[0] value=aula-a clock=A:1,B:0,C:0

[1] value=aula-b clock=A:0,B:1,C:0

Questa scelta protegge una proprietà di safety:

non perdere silenziosamente un aggiornamento concorrente

Comando	Contratto
GET	valore, not found o conflitto
SET	scrittura semplice senza conflitto locale
SYNC	anti-entropy bidirezionale
RESOLVE	risoluzione esplicita dei siblings
DELETE	tombstone versionata
DUMP	stato locale in JSON

Il contratto distingue scrittura, sincronizzazione e scelta applicativa.

SET room aula-a

Percorso implementativo:

- 1 legge le versioni locali della chiave;
- 2 se esistono siblings, rifiuta;
- 3 calcola il merge dei clock locali;
- 4 incrementa la componente della replica;
- 5 salva la nuova versione;
- 6 compatta le versioni dominate.

SET Non Risolve Conflitti

Se esistono siblings:

```
A> SET room aula-c
```

```
A< ERR conflict_exists key=room use RESOLVE <key> <value>
```

Motivo:

- una scrittura semplice non dichiara quale conflitto sta resolvendo;
- scegliere un vincitore è una decisione applicativa;
- nascondere il conflitto renderebbe il contratto ambiguo.

Comando GET

GET room

Risposte possibili:

NOT_FOUND key=room

OK key=room value=aula-a clock=A:1,B:0,C:0 origin=A

CONFLICT key=room siblings=2 | ...

GET non finge che il conflitto non esista.

SYNC 6482

Passi logici:

- ➊ il nodo locale chiede lo snapshot al peer;
- ➋ fonde lo snapshot remoto;
- ➌ invia il proprio snapshot aggiornato al peer;
- ➍ il peer fonde a sua volta;
- ➎ entrambi eliminano solo versioni dominate.

SYNC non è consenso: scambia e compatta stato.

Dato un conflitto:

room:

aula-a clock=A:1,B:0,C:0

aula-b clock=A:0,B:1,C:0

La risoluzione:

RESOLVE room aula-c

-> clock=A:2,B:1,C:0

Il nuovo clock domina entrambi i siblings.

DELETE Come Tombstone

Una cancellazione distribuita non è solo rimozione locale.

Nel laboratorio DELETE crea una tombstone versionata:

```
value="" deleted=True clock=...
```

Motivo:

- una cancellazione può essere concorrente con una scrittura;
- serve un clock per confrontarla causalmente;
- rimuovere subito la traccia può far riapparire valori vecchi.

Nel file `vector_clock.py`:

`compare(left, right) -> before | after | same | concurrent`

`merge(left, right) -> massimo componente per componente`

`increment(clock, node_id) -> incrementa solo node_id`

Queste funzioni sono piccole, ma definiscono la semantica causale del sistema.

Idea implementativa:

```
for candidate in versions:
    if exists other such that candidate.clock < other.clock:
        discard candidate
    else:
        keep candidate
```

Conseguenza:

- versioni vecchie vengono eliminate;
- versioni concorrenti restano;
- la safety dipende dalla correttezza di questo predicato.

Demo: Scrittura Causale

A> SET course sistemi-distribuiti

A< OK ... clock=A:1,B:0,C:0

B> GET course

B< NOT_FOUND key=course

A> SYNC 6482

B> GET course

B< OK ... clock=A:1,B:0,C:0

Il valore non è globale finché non viene propagato.

Demo: Scritture Concorrenti

A> SET room aula-a

A< OK ... clock=A:1,B:0,C:0

B> SET room aula-b

B< OK ... clock=A:0,B:1,C:0

A> SYNC 6482

A> GET room

A< CONFLICT key=room siblings=2 | ...

La concorrenza emerge solo quando le repliche si scambiano stato.

Demo: Risoluzione E Convergenza

```
A> RESOLVE room aula-c
A< OK resolved ... clock=A:2,B:1,C:0

A> SYNC 6482
B> GET room
B< OK ... value=aula-c clock=A:2,B:1,C:0

B> SYNC 6483
C> GET room
C< OK ... value=aula-c clock=A:2,B:1,C:0
```

La versione risolta domina i siblings e può propagarsi.

Proprietà principale:

una versione concorrente non viene eliminata durante la sincronizzazione

Errori che violerebbero safety:

- trattare l'ultimo messaggio ricevuto come vincitore automatico;
- confrontare clock con una somma delle componenti;
- cancellare tombstone troppo presto;
- usare SET come risoluzione implicita.

Obiettivo operativo:

se le repliche continuano a sincronizzarsi e i conflitti vengono risolti, il sistema converge

Ipotesi necessarie:

- rete prima o poi disponibile;
- repliche raggiungibili;
- sincronizzazioni ripetute;
- conflitti risolti dall'applicazione;
- membership coerente.

Ruolo	Responsabilità
Driver A	comandi sulla replica A
Driver B	comandi sulla replica B
Driver C	comandi sulla replica C
Osservatore clock	annota i vector clock
Verificatore contratto	controlla risposte e vincoli
Analista safety/liveness	identifica proprietà e rischi

Il laboratorio funziona bene se i ruoli sono separati.

Il laboratorio non implementa:

- persistenza su disco;
- autenticazione;
- membership dinamica;
- consenso;
- quorum;
- garbage collection avanzata delle tombstone;
- merge automatico applicativo;
- CRDT.

Sono limiti intenzionali: isolano il ruolo dei vector clock.

- Un clock fisico avrebbe risolto il conflitto?
- Perché SET viene rifiutato in presenza di siblings?
- Chi deve decidere il valore vincente?
- Quale proprietà di safety protegge il mantenimento dei siblings?
- Quale ipotesi di liveness serve per arrivare alla convergenza?
- Quando servirebbe consenso invece di vector clock?